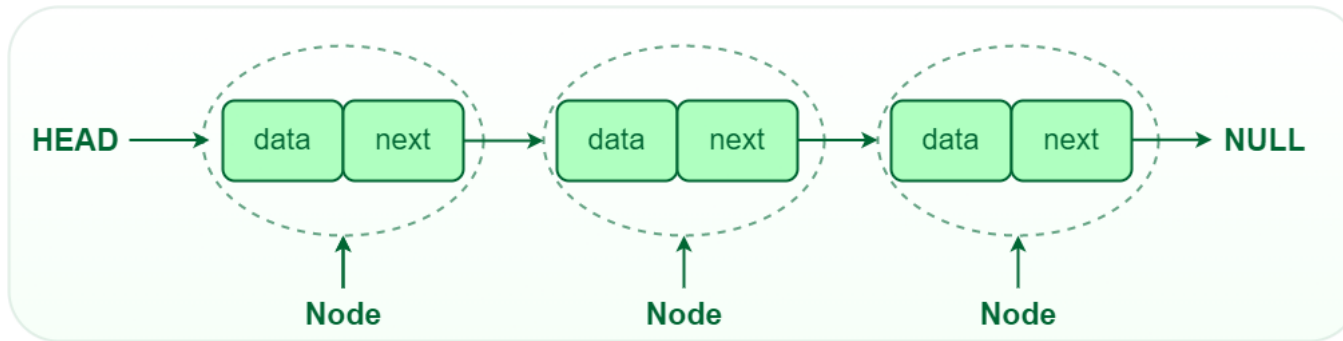


Lecture 11

Linked Lists II



Announcements

- ❖ **Office hours** are up on course website
 - T: 12pm-4pm; F: 11am-1pm
 - Schedule an appointment individually or as a group
- ❖ **Project 1** Evaluation deliverable due **Friday @ 11:59pm**
 - See [example project](#) on Github
 - You received feedback via email
- ❖ **Midterm Exam** is next Tuesday (10/6) during lecture
 - 85% on paper (~40 minutes); 15% on computer (due at midnight)
 - Study guide posted on Discord
 - Lab next Monday is a review exercise

Last time

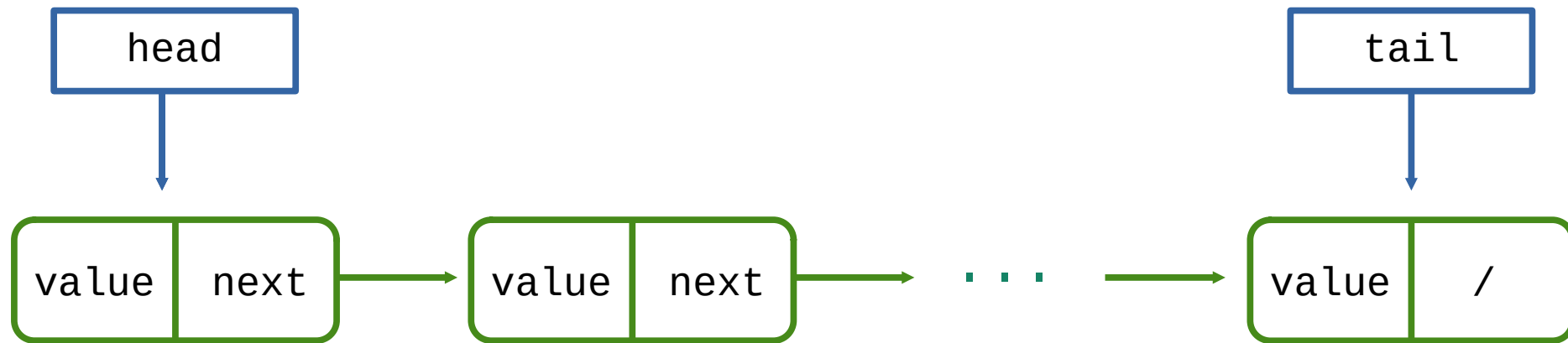
- ❖ Introduce new data structure: `linked list`

Today

- ❖ Tradeoffs between arrays and linked lists
- ❖ Implementing a queue with a linked list
- ❖ Doubly linked lists
- ❖ Discuss midterm study guide

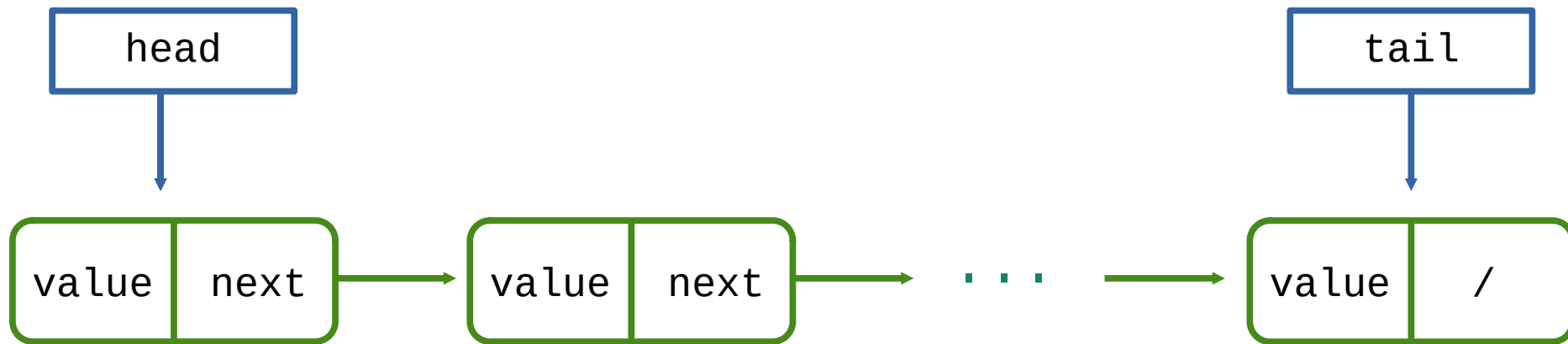
Linked List

- ❖ A chain of nodes
- ❖ Property **head** pointing to the first node in the chain
- ❖ Property **tail** pointing to the last node in the chain



Linked List Example

- ❖ Document you are editing
- ❖ `head` is like a blank file and each node applies a diff to the document
- ❖ `tail` is like the current state of the document



Linked List Operations

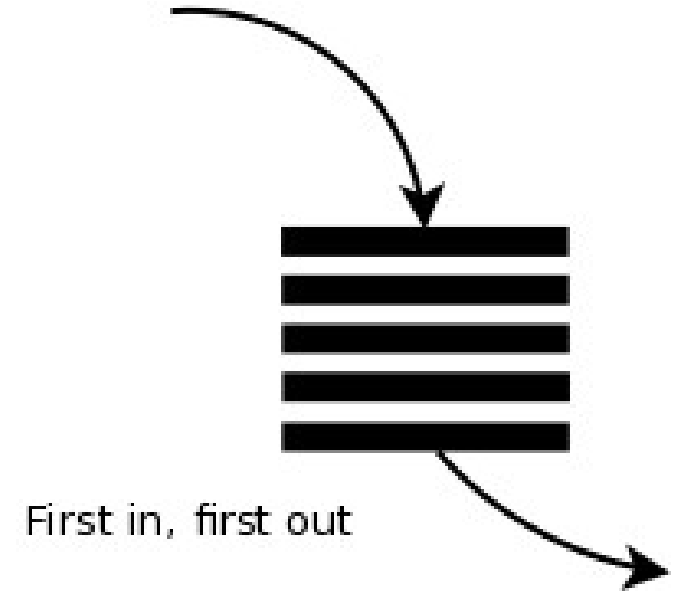
- ❖ Getting an index value: $O(n)$
- ❖ Insertion or deletion after the current node: $O(1)$
- ❖ **Challenge:** getting to the current node may cost $O(n)$
 - Starting at head, insertion or deletion at an arbitrary node is $O(n)$

Array vs. Linked List

	Array	Linked List
Getting at index	$O(1)$	$O(n)$
Inserting at index	$O(n)$	$O(n)$
Deleting at index	$O(n)$	$O(n)$
Inserting after current index	$O(n)$	$O(1)$
Inserting before current index	$O(n)$	$O(n)$
Appending to end	$O(1)$	$O(1)$
Deleting index 0	$O(n)$	$O(1)$

Queue

- ❖ An abstract data type that holds objects in order and can
 - Enqueue
 - Dequeue
 - Peek



Implementing a Queue

Strategy: build a queue out of data structures we know

- ❖ Implement a queue using an array (Python's list)
 - Holds objects in order and can enqueue, dequeue, and peek
 - However: dequeue are linear time operations

Implementing a Queue

Strategy: build a queue out of data structures we know

- ❖ Implement a queue using an array (Python's list)
 - Holds objects in order and can enqueue, dequeue, and peek
 - However: dequeue are linear time operations
- ❖ Implement a queue using a linked list

Implementing a Queue

Strategy: build a queue out of data structures we know

- ❖ Implement a queue using an array (Python's list)
 - Holds objects in order and can enqueue, dequeue, and peek
 - However: dequeue are linear time operations
- ❖ Implement a queue using a linked list
 - Peek is checking the value of the head node: $O(1)$
 - Enqueue is inserting after the tail node: $O(1)$
 - Dequeue is deleting the head node: $O(1)$

Implementing a Queue

Strategy: build a queue out of data structures we know

- ❖ Implement a queue using an array (Python's list)
 - Holds objects in order and can enqueue, dequeue, and peek
 - However: dequeue are linear time operations
 - ❖ Implement a queue using a linked list
 - Peek is checking the value of the head node: $O(1)$
 - Enqueue is inserting after the tail node: $O(1)$
 - Dequeue is deleting the head node: $O(1)$
- } efficient implementation

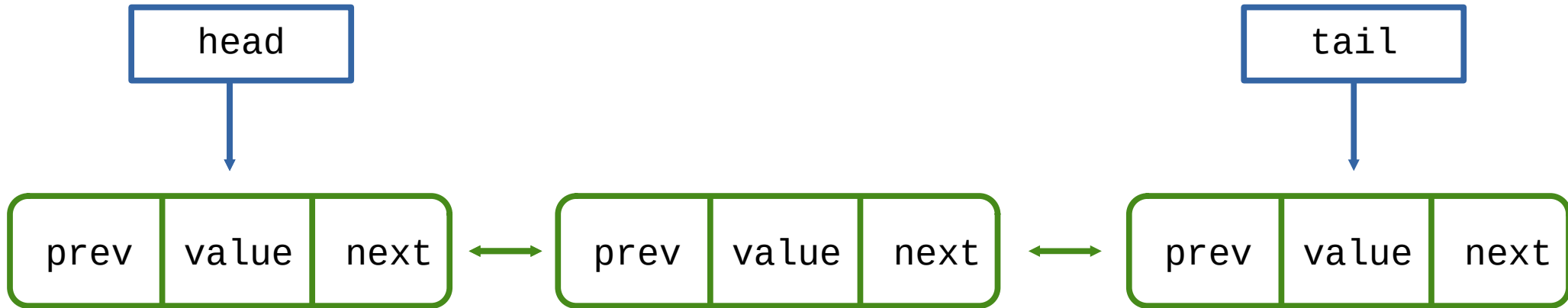
Doubly Linked List

- ❖ Idea: add a backwards pointer
- ❖ Data are stored in **nodes** that have three properties"
 - value the item held by the list
 - next points to another node
 - prev points to prior node



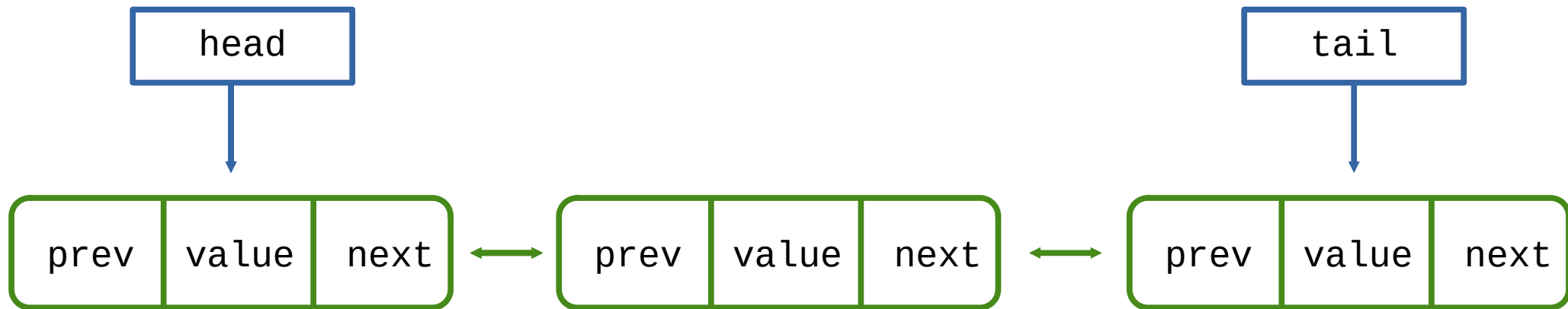
Doubly Linked List

- ❖ A chain of nodes
- ❖ Property **head** pointing to the first node in the chain
- ❖ Property **tail** pointing to the last node in the chain



Doubly Linked List Example

- ❖ Document you are editing, where head is a blank file and tail is the current
- ❖ You can start from head and apply changes
- ❖ Or start from tail and undo changes



Array vs. Linked List vs. DLL

	Array	Linked List	Doubly Linked List
Getting at index	$O(1)$	$O(n)$	$O(n)$
Inserting at index	$O(n)$	$O(n)$	$O(n)$
Deleting at index	$O(n)$	$O(n)$	$O(n)$
Inserting after current index	$O(n)$	$O(1)$	$O(1)$
Inserting before current index	$O(n)$	$O(n)$	$O(1)$
Appending to end	$O(1)$	$O(1)$	$O(1)$
Deleting index 0	$O(n)$	$O(1)$	$O(1)$

Takeaways

- ❖ Linked List is a new list data structure
- ❖ Use them to efficiently implement a queue
- ❖ Doubly linked lists offer more efficient operations at the cost of more storage

Next Time

- ❖ Midterm Exam
- ❖ Then, Fall Break!